

情報学群実験第 4 実験レポート

学籍番号 1280391

細川 夏風

2026 年 7 月 5 日

1 目的

近年の急速な情報技術の発展により，現実世界以上に活動の幅を広げることが増えた．その最たる例として Virtual Reality (VR) が挙げられる．VR には制限が極端に少ないように感じる．しかし，その実は多くの制限が存在する．それは多くがヒト自体の肉体的制限や特性に依るものであることが多い．今実験では，そのヒトの制限や特性を理解し，より没入感のある VR 体験を提供するための検証を行う．

2 方法

本実験は，視覚探索課題を行う心理物理実験の実施と，得られたデータに対する回帰分析および統計的仮説検定の 2 つの段階に分けて実施した．

次実験の VR ゲーム作成では，Unity を用いた 3D ゲームの開発と，Meta Quest 3S および Meta XR SDK を用いたバーチャルリアリティ（VR）環境への対応という 2 つの段階で実施した．

2.1 心理物理実験の実施（視覚探索課題）

実験プログラムは Python および Psychopy を用いて作成された `visualsearch.py` を使用した [付録 1]．実験参加者は，ディスプレイまでの観察距離を一定に保ち，画面上に提示される妨害刺激の中に目標刺激が存在するか否かを可能な限り早く，かつ正確に判断する課題を行った．刺激の条件として，目標刺激には「丸」および「丸＋棒」の 2 種類を用いた．目標刺激が「丸」のときの妨害刺激は「丸＋棒」であり，目標刺激が「丸＋棒」のときの妨害刺激は「丸」とした．提示される刺激の総数（セットサイズ）は 4, 8, 16 個のいずれかであり，目標刺激が 1 つ存在する試行と，目標刺激がなく妨害刺激のみの試行がランダムな順序で提示された．参加者は，目標刺激がある場合は「J」キーを，ない場合は「F」キーを押して回答した．実験は 12 試行の練習試行を行った後，240 試行の本実験を実施した．なお，課題の処理自体にかかる時間以外の要因を排除するため，反応に 5 秒以上かかった時間オーバーの試行および不正解だった試行のデータは分析から除外した．

2.2 データ分析および統計的仮説検定

心理物理実験で得られた各自の反応時間データについて、目標刺激の種類，有無，およびセットサイズごとに平均値を算出した後，複数人数分のデータを取りまとめた．まとめたデータに基づき，各条件における反応時間の平均値，不偏分散，標準偏差，および標準誤差を算出した．次に，横軸をセットサイズ，縦軸を反応時間とする散布図を作成し，各条件の平均値のプロットと標準誤差を示す誤差線を付加した．得られたプロットに対して単回帰分析を行い，セットサイズに対する反応時間の回帰直線の傾きを算出した．最後に，目標刺激が存在する条件において，目標刺激が「丸」のときと「丸+棒」のときで回帰直線の傾きの平均値に統計的な有意差があるかを確認するため，対応のある t 検定を用いた統計的仮説検定を実施した．

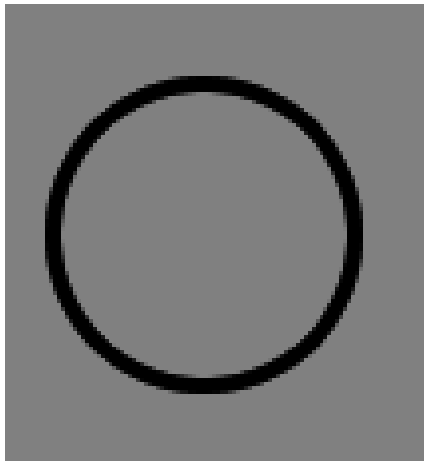


図 1 丸の目標刺激

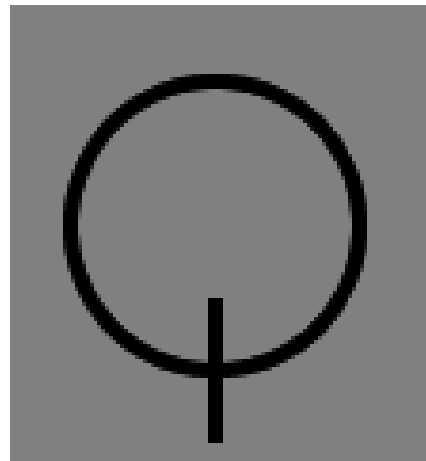


図 2 丸+棒の目標刺激

2.3 VR 実験環境および使用機器

本実験で使用した主な環境および機器・ソフトウェアは以下の通りである．

- 心理物理実験（視覚探索課題）
 - PC
 - Python および Psychopy（バージョン 2026.1.3）
- 3D ゲームおよび VR コンテンツの開発
 - VR 対応ゲーミング PC
 - Meta Quest 3S ヘッドセット
 - Meta Link ケーブル（または USB-C ケーブル）
 - Unity 6（エディターバージョン 6000.1.11f1）
 - Meta Horizon Link アプリ
 - Meta XR All-in-One SDK
 - Visual Studio（C#スクリプト作成・編集・デバッグ用）

2.4 3D ゲームの基礎開発

Unity 6 を用いて、「ボールを転がしてコインを集める」3D ゲームの開発を行った。まず、Cube を利用して床と壁を作成し、Sphere を用いてプレイヤーとなるボールを作成した。プレイヤーには Rigidbody コンポーネントを追加し、物理演算を適用した。次に、C#スクリプトを作成し、FixedUpdate 関数内で Rigidbody.AddForce を用いることで、キーボード入力によるプレイヤーの移動を実装した [付録 2]。また、プレイヤーに追従してカメラが移動するスクリプト [付録 3] や、Cylinder で作成したコインを回転させるスクリプト [付録 4] を作成した。さらに、プレイヤーがコインのタグに触れた際にコインを非表示にし、TextMeshPro を用いて画面上にスコアと経過時間（タイマー）を表示する機能を実装した。最後に、作成したシーンをテンプレートとして保存し、複数のステージを作成した。全コイン獲得後に特定のキーを入力することで、次のステージへ遷移する処理も実装した。

2.5 VR 環境への対応とインタラクションの実装

作成した 3D ゲームを VR 環境で体験できるようにするため、Meta Quest 3S ヘッドセットと PC を Quest Link を用いて接続した。Unity プロジェクト内で XR Plugin Management を有効にし、Meta XR All-in-One SDK を導入した。その後、Building Blocks 機能から Camera Rig をシーンに追加し、VR 空間での視点を構築した。また、Canvas のレンダーモードを World Space に変更し、VR 空間内にスコア等の UI が表示されるように調整した。さらに、Grab Interaction などの機能を導入し、キーボードではなくハンドトラッキングを用いてゲーム内のボールを手でつかんで傾けることで、ボールを転がしてコインを集められるように操作方法を変更した。

2.6 ビリヤードゲームの作成

最終的に自班ではビリヤードゲームの作成を行った。プレイヤーは、開始時に目の前にある棒を手で掴みゲームを始める。プレイヤーはビリヤード台に配置されている 6 つの番号つきボールを棒で転がした白いボールですべて穴に行けることによってゲームが終了する。ゲーム終了時は、スコアと経過時間が表示される。このとき、穴の下にコインを配置し、ボールが穴に入るとコインが獲得できる仕組みになっている。ゲーム中はプレイヤーの持つコントローラーの左スティックを傾けることで移動が可能になっている。このゲームはビリヤードゲームであるがルールに関しては簡略化されており、ボールを穴に入れる順番は問わない。また、この時使用した Asset は Unity Asset Store で入手したものを付録に示しておく [付録 B]。

3 結果

3.1 統計分析

視覚探索課題における反応時間について、条件ごとおよびセットサイズごとの平均値を算出した結果を表1に示す。

表1 各条件における反応時間の平均値 (ms)

目標刺激	目標刺激の有無	セットサイズ 4	セットサイズ 8	セットサイズ 16
丸	あり	773	886	950
丸	なし	791	931	1046
丸+棒	あり	684	672	682
丸+棒	なし	702	700	731

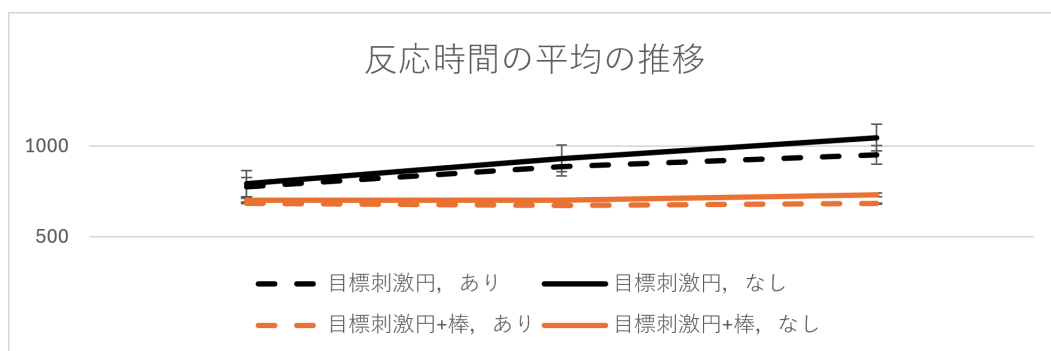


図3 各条件におけるセットサイズと反応時間の関係

表2 目標刺激あり条件における回帰直線の傾きの t 検定結果

t 値	p 値
3.02	0.0165

また、目標刺激が存在する条件（あり条件）において、目標刺激が「丸」のときと「丸+棒」のときで回帰直線の傾きの平均値に有意な差があるかを確認するため、対応のある t 検定を行った。その結果、 t 値は約 3.02, p 値は約 0.0165 となり、有意水準 5%において統計的に有意な差が認められた。

3.2 VR ゲームの作成

Unity および Meta Quest 3S を用いた VR コンテンツ開発の結果として、ビリヤードゲームを作成した。本コンテンツは、初期に開発した「ボールを転がしてコインを集める」ゲームの知見を発展させたものであり、VR 空間内にビリヤード台や複数の的球、キューなどを配置して構築した。また、タイマー機能やスコア機能も実装している。Meta XR SDK の機能を活用することで、プレイヤーは VR 環境においてハンドトラッキング等のインタラクションを用いて実際のビリヤードのような操作を体験することができる。作成したビリヤードゲームの Unity での開発画面を図 4 に、VR 環境でのプレイ画面を図 5 に示す。

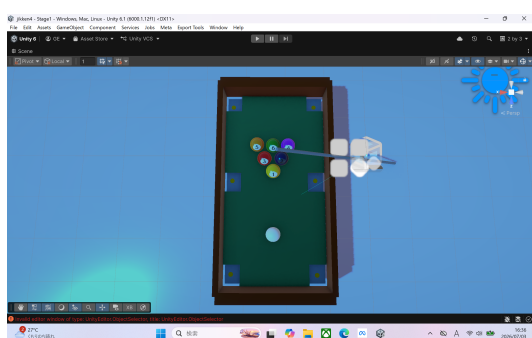


図 4 Unity でのゲーム開発画面

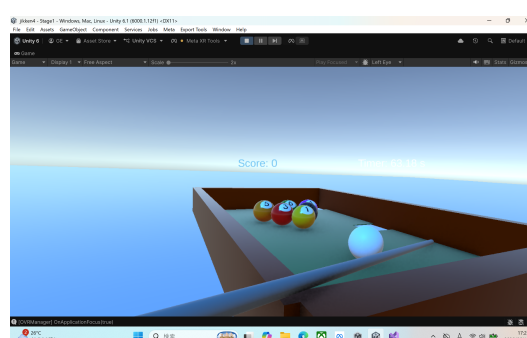


図 5 VR 環境でのプレイ画面

4 考察

4.1 視覚探索課題における反応時間の傾向

本実験結果から、目標刺激の条件によってセットサイズに対する反応時間の増加率に顕著な違いが見られた。目標刺激が「丸+棒」の条件では、セットサイズが増加しても反応時間はほぼ一定であった。一方、目標刺激が「丸」の条件では、セットサイズが増加するのに比例して反応時間が顕著に増加した。また、対応のある t 検定の結果からも、2 条件の回帰直線の傾きには有意な差があることが示された。加えて、目標刺激ありの条件に対して、なしの条件の方が全体的に反応時間が長くなる傾向も確認された。

4.2 特徴統合理論と探索非対称性による解釈

これらの結果は、Treisman らが提唱した特徴統合理論 [1] および視覚探索の非対称性の観点から説明できると考えられる。妨害刺激である「丸」の中に、目標刺激「丸+棒」が存在する場合、「棒」という初期的な視覚特徴が前注意的段階で素早く検出され、ポップアウト現象が生じる。この場合、視覚情報は並列的に処理されるため、セットサイズに依存せず短い反応時間で目標を検出できたと推測できる。

これに対し、妨害刺激である「丸＋棒」の中に、目標刺激「丸」が存在する場合、目標刺激は「棒がない」という特徴の欠如によって定義される。特徴統合理論によれば、特徴の欠如はポップアウトを引き起こさないため、注意を各刺激に順番に向ける直列的な処理を強いられる。その結果、セットサイズが大きくなるほど確認すべき項目が増加し、反応時間が線形に増加したと考えられる。

4.3 目標刺激の有無による違いについて

目標刺激なしの条件で反応時間が長くなった理由についても、直列処理の性質から説明が可能である。目標刺激が存在する場合は、探索の途中で目標を発見した時点で探索を終了できる自己終了型探索であるのに対し、目標刺激が存在しない場合は、画面上のすべての刺激に対して探索を完了する網羅的探索を行わなければ「ない」と判断できないためである。

以上のことから、ヒトの視覚特性には対象の特徴の有無によって処理効率が変化する非対称性が存在することが確認できた。このようなヒトの認知制限や特性を理解することは、VR空間内において重要な情報をユーザーに見落とさせないよう効果的に提示するなど、認知負荷を考慮した設計を行う上で有用であると言える。

参考文献

- [1] 脳科学辞典「注意のモデル」, 特徴統合理論,
<https://bsd.neuroinf.jp/wiki/%E6%B3%A8%E6%84%8F%E3%81%AE%E3%83%A2%E3%83%87%E3%83%AB>
(2026年7月5日閲覧)

A プログラムコード

心理物理実験で使用したプログラム `visualearch.py` のソースコードを以下に示す。

Listing 1 `visualearch.py`

```
1  #!/usr/bin/env python
2  """
3  実験概要:
4      - ターゲット（奇数刺激）の有無を判断する視覚探索課題
5      - 刺激: 円 vs 円＋棒
6      - 反応キー: F（ターゲットあり）、J（ターゲットなし）
7      - セットサイズ: 4, 8, 16
8  """
9
10 import csv
```

```

11 import os
12 import sys
13 from psychopy import visual, core, event, monitors, gui
14 import numpy as np
15
16
17 # 実験パラメタ
18
19 dlg_dict = {'学籍番号': ''}
20 dlg = gui.DlgFromDict(dictionary=dlg_dict, title='学籍番号を入力してくだ
    さい', order=['学籍番号'])
21 if not dlg.OK:
22     core.quit()
23 SUBJECT = (dlg_dict.get('学籍番号|0') or dlg_dict.get('学籍番号') or '').
    strip()
24 if SUBJECT == '':
25     core.quit()
26 SET_SIZES = [4, 8, 16] # 刺激セットサイズ
27 N_SET_SIZES = len(SET_SIZES)
28 P_TRIALS = 12 # 練習試行数
29 R_TRIALS = 20 # 繰り返し数
30 X_TRIALS = R_TRIALS * 2 * 2 * N_SET_SIZES # 本実験試行数
31
32 MAX_WAIT = 5.0 # 反応時間の制限時間（秒）
33 FIXATION_DURATION = 0.5 # 注視点の表示時間（秒）
34
35 # 刺激描画パラメタ（ピクセル単位）
36 N_COLUMNS = 5
37 N_ROWS = 5
38 STROKE = 4 # 線の太さ（px）
39 MARGIN = 4 # 円のマージン（px）
40
41
42 # モニター設定（解像度は自動取得）
43
44 mon = monitors.Monitor('testMonitor')

```

```

45
46
47 # ウィンドウ作成（フルスクリーン）
48
49 win = visual.Window(
50     fullscr=True,
51     monitor=mon,
52     units='pix',
53     color=[0, 0, 0],
54     colorSpace='rgb',
55     allowGUI=False,
56     waitBlanking=True,
57 )
58 win.mouseVisible = False
59
60 # 実際の解像度を取得
61 SCREEN_W, SCREEN_H = win.size
62
63 # グレー（PsychoPy rgb スケール）
64 GRAY = [0, 0, 0]
65 BLACK = [-1, -1, -1]
66 WHITE = [1, 1, 1]
67
68
69 # 刺激レイアウト計算（画面高さの8割の正方形に収まるよう動的計算）
70
71 FIELD_SIZE = int(SCREEN_H * 0.7) # 刺激提示領域のサイズ（正方形）
72 STIMULUS_RANGE = FIELD_SIZE // N_COLUMNS # 1cellのサイズ
73 OBJECT_SIZE = int(STIMULUS_RANGE // 2 * 0.8) # 刺激サイズ（cellの半分の80%）
74 JITTER = int(OBJECT_SIZE * 0.8) # cellの中心からのバラツキ
75
76 col_centers = (np.arange(N_COLUMNS) + 0.5) * STIMULUS_RANGE
77 row_centers = (np.arange(N_ROWS) + 0.5) * STIMULUS_RANGE
78 XX, YY = np.meshgrid(col_centers, row_centers)

```

```

79
80 XX = XX - FIELD_SIZE / 2
81 YY = FIELD_SIZE / 2 - YY
82
83 N_POSITIONS = N_COLUMNS * N_ROWS
84 grid_positions = np.column_stack([XX.flatten(), YY.flatten()])
85
86
87 # 刺激オブジェクト生成関数
88
89
90 def make_circle_stim(win, pos=(0, 0)):
91     """円のみ刺激"""
92     r = (OBJECT_SIZE - MARGIN * 2) / 2
93     return visual.Circle(
94         win,
95         radius=r,
96         pos=pos,
97         lineColor=BLACK,
98         fillColor=None,
99         lineWidth=STROKE,
100         units='pix',
101     )
102
103 def make_cplusline_stim(win, pos=(0, 0)):
104     """円＋棒刺激
105     """
106     r = (OBJECT_SIZE - MARGIN * 2) / 2
107     circle = visual.Circle(
108         win,
109         radius=r,
110         pos=pos,
111         lineColor=BLACK,
112         fillColor=None,
113         lineWidth=STROKE,
114         units='pix',

```

```

115     )
116     # 棒の中点を円弧上（6時方向）に置く
117     # 棒の長さ = r、中点 = (pos[0], pos[1] - r)
118     half = r / 2
119     line = visual.Line(
120         win,
121         start=(pos[0], pos[1] - r + half),    # 上端: 円弧より half だけ上
122         end= (pos[0], pos[1] - r - half),    # 下端: 円弧より half だけ下
123         lineColor=BLACK,
124         lineWidth=STROKE,
125         units='pix',
126     )
127     return circle, line
128
129 def make_fixation(win):
130     """十字の注視点"""
131     h_line = visual.Line(win, start=(-OBJECT_SIZE/2, 0), end=(OBJECT_SIZE
132         /2, 0),
133         lineColor=BLACK, lineWidth=STROKE, units='pix')
134     v_line = visual.Line(win, start=(0, -OBJECT_SIZE/2), end=(0,
135         OBJECT_SIZE/2),
136         lineColor=BLACK, lineWidth=STROKE, units='pix')
137     return h_line, v_line
138
139 # キー待ちヘルパー（ESCで即終了）
140
141 def wait_key():
142     """任意キー待ち。ESCが押された場合はデータファイルを閉じて終了。"""
143     keys = event.waitKeys()
144     if keys and 'escape' in keys:
145         try:
146             data_file.close()
147         except Exception:
148             pass
149     win.close()

```

```

149         core.quit()
150
151
152 # データファイルの設定
153
154 DATA_FILENAME = f'data{SUBJECT}.csv'
155 write_header = not os.path.exists(DATA_FILENAME)
156
157 data_file = open(DATA_FILENAME, 'a', newline='', encoding='utf-8')
158 writer = csv.writer(data_file)
159 if write_header:
160     writer.writerow(['subject', 'practice/real', 'trial#',
161                     'target/nontarget', 'target_type', 'setsize',
162                     'error', 'responsetime_ms'])
163
164 def write_trial(subject, block, trial, target, whichtarget, setsize, err,
165               rt_ms):
166     writer.writerow([subject, block, trial, target, whichtarget, setsize,
167                     err, rt_ms])
168     data_file.flush()
169
170 # 注視点
171
172 fix_h, fix_v = make_fixation(win)
173
174 # 教示文の表示
175
176 instructions = [
177     "画面に表示される図形の中に、他と異なる目標刺激（ターゲット）があるか",
178     "どうかを判断してください。",
179     "ターゲットがある場合は、右手人差し指で 'J' キーを押してください。",
180     "ターゲットがない場合は、左手人差し指で 'F' キーを押してください。",
181     "できるだけ正確に、かつ素早く反応してください。",
182     f"練習試行数は {P_TRIALS} 回、本実験の試行数は {X_TRIALS} 回です。",

```

```

182     "",
183     "準備ができれば、任意のキーを押して開始してください。",
184 ]
185 instr_stims = []
186 line_height = 40
187 start_y = (len(instructions) - 1) * line_height / 2
188 for i, line in enumerate(instructions):
189     stim = visual.TextStim(
190         win,
191         text=line,
192         pos=(0, start_y - i * line_height),
193         color=BLACK,
194         height=36,
195         units='pix',
196         wrapWidth=SCREEN_W * 0.85,
197     )
198     instr_stims.append(stim)
199
200 win.flip()
201 for stim in instr_stims:
202     stim.draw()
203 win.flip()
204 wait_key()
205 core.wait(1.0)
206
207
208 # 実験ループ
209
210 clock = core.Clock()
211
212 for block in range(1, 3):
213     if block == 1:
214         block_label = 'Practice'
215         n_trials = P_TRIALS
216     else:
217         block_label = 'Test'

```

```

218         n_trials = X_TRIALS
219
220     # ブロック開始メッセージ
221     msg = visual.TextStim(
222         win,
223         text=f'Press any Key to start\n({block_label} = {n_trials} trials
224             )',
225         pos=(0, 0),
226         color=BLACK,
227         height=36,
228         units='pix',
229     )
230     msg.draw()
231     win.flip()
232     wait_key()
233     core.wait(0.5)
234
235     # 試行順序のシャッフル（本試行のみ使用）
236     trial_sequence = np.random.permutation(n_trials) + 1 # 1-indexed
237
238     for trial in range(1, n_trials + 1):
239         win.flip() # 画面クリア
240
241         # 刺激条件の設定
242
243         if block == 1:
244             # 練習試行：ランダム
245             target = np.random.randint(1, 3) # 1=あり, 2=なし
246             whichtarget = np.random.randint(1, 3) # 1=円がターゲット,
247                 2=円+棒がターゲット
248             set_size = SET_SIZES[np.random.randint(0, N_SET_SIZES)]
249         else:
250             # 本試行：MATLABコードと同じロジックを再現
251             ts = trial_sequence[trial - 1]
252             target = (ts - 1) % 2 + 1

```

```

252     whichtarget = int(np.ceil(ts / (2 * N_SET_SIZES * R_TRIALS)))
253     set_size     = SET_SIZES[int(np.ceil(((ts - 1) % (2 *
        N_SET_SIZES * R_TRIALS) + 1) / (2 * R_TRIALS))) - 1]
254
255     # 位置のランダム配置
256     position_index = np.random.permutation(N_POSITIONS)
257
258
259     # 刺激の生成
260
261     stim_objects = []    # (stim_type, pos) を格納: stim_type = '
        circle' or 'cplusline'
262
263     for i in range(set_size):
264         pos_idx = position_index[i]
265         base_x, base_y = grid_positions[pos_idx]
266
267         # ジッター: MATLABの round(ceil(jitter*rand) - jitter/2) を再
            現
268         dx = round(np.random.randint(1, JITTER + 1) - JITTER / 2)
269         dy = round(np.random.randint(1, JITTER + 1) - JITTER / 2)
270         pos = (base_x + dx, base_y + dy)
271
272         # どの刺激を描くか決定
273         if whichtarget == 1:    # 円がターゲット
274             if target == 1 and i == 0:
275                 item_type = 'circle'
276             else:
277                 item_type = 'cplusline'
278         else:                    # 円+棒がターゲット
279             if target == 1 and i == 0:
280                 item_type = 'cplusline'
281             else:
282                 item_type = 'circle'
283
284         stim_objects.append((item_type, pos))

```

```

285
286
287     # 注視点提示
288
289     fix_h.draw()
290     fix_v.draw()
291     win.flip()
292     core.wait(FIXATION_DURATION)
293
294
295     # 刺激提示
296
297     for item_type, pos in stim_objects:
298         if item_type == 'circle':
299             s = make_circle_stim(win, pos=pos)
300             s.draw()
301         else:
302             c, l = make_cplusline_stim(win, pos=pos)
303             c.draw()
304             l.draw()
305     event.clearEvents() # 注視点提示中のキー入力を破棄
306     win.flip()
307
308
309     # 反応取得
310
311     clock.reset()
312     response = None
313     rt_ms = 0
314     err = None
315
316     while response is None and clock.getTime() < MAX_WAIT:
317         keys_pressed = event.getKeys(
318             keyList=['f', 'j', 'escape'],
319             timeStamped=clock,
320             )

```

```

321         if keys_pressed:
322             key, t = keys_pressed[0]
323             response = key
324             rt_ms = round(t * 1000)
325
326
327     # 正誤判定
328
329     if response is None:
330         err = 3
331         feedback_text = 'Time out!'
332         rt_ms = 0
333     elif response == 'escape':
334         data_file.close()
335         win.close()
336         core.quit()
337     elif response == 'f':
338         response_key = 2    # f = ターゲットなし
339         if response_key == target:
340             err = 0
341             feedback_text = 'Correct!'
342         else:
343             err = 1
344             feedback_text = 'Wrong!'
345     elif response == 'j':
346         response_key = 1    # j = ターゲットあり
347         if response_key == target:
348             err = 0
349             feedback_text = 'Correct!'
350         else:
351             err = 1
352             feedback_text = 'Wrong!'
353
354
355     # データ書き込み
356

```

```

357         write_trial(SUBJECT, block, trial, target, whichtarget, set_size,
358                     err, rt_ms)
359
360     # フィードバック表示
361
362     fb_stim = visual.TextStim(
363         win,
364         text=f'{{feedback_text}} / RT = {{rt_ms}} ms',
365         pos=(0, 0),
366         color=BLACK,
367         height=36,
368         units='pix',
369     )
370     fb_stim.draw()
371     win.flip()
372     core.wait(1.0)
373
374
375 # 終了処理
376
377 data_file.close()
378
379 end_stim1 = visual.TextStim(win, text='All done! Thank you for
380     participating',
381                             pos=(0, 40), color=BLACK, height=36, units='
382                             pix')
383 end_stim2 = visual.TextStim(win, text='Press any key',
384                             pos=(0, 0), color=BLACK, height=36, units='
385                             pix')
386
387 end_stim1.draw()
388 end_stim2.draw()
389 win.flip()
390 wait_key()
391
392 win.close()

```

389 | `core.quit()`

VR ゲーム開発で作成した C#スクリプトのソースコードを以下に示す.

Listing 2 MovePlayer.cs

```
1 using UnityEngine;
2 using TMPro;
3 using UnityEngine.SceneManagement;
4
5 public class MovePlayer : MonoBehaviour
6 {
7     private Rigidbody rb;
8     public float forceMagnitude;
9
10    public TextMeshProUGUI pointsText;
11    public TextMeshProUGUI timerText;
12
13    public static int points;
14    private int totalCoins;
15    private float timer;
16
17    // Start is called once before the first execution of Update after
18    // the MonoBehaviour is created
19    void Start()
20    {
21        rb = GetComponent<Rigidbody>();
22        points = 0;
23        totalCoins = GameObject.FindGameObjectsWithTag("Coin").Length;
24    }
25
26    // Update is called once per frame
27    void Update()
28    {
29        if (points >= totalCoins)
```

```

30         pointsText.text = "Score: " + points.ToString() + "\nLevel
           Completed!";
31         if (Input.GetKeyDown(KeyCode.Space))
32         {
33             LoadNextScene();
34         }
35     }
36     else
37     {
38         pointsText.text = "Score: " + points.ToString();
39         timer += Time.deltaTime;
40         timerText.text = "Timer: " + timer.ToString("F2") + " s";
41     }
42 }
43
44 private void FixedUpdate()
45 {
46     if (Input.GetKey(KeyCode.W))
47     {
48         rb.AddForce(new Vector3(0f, 0f, 1f) * forceMagnitude); // W:
           forward
49     }
50     if (Input.GetKey(KeyCode.S))
51     {
52         rb.AddForce(new Vector3(0f, 0f, -1f) * forceMagnitude); // S:
           backward
53     }
54     if (Input.GetKey(KeyCode.A))
55     {
56         rb.AddForce(new Vector3(-1f, 0f, 0f) * forceMagnitude); // A:
           left
57     }
58     if (Input.GetKey(KeyCode.D))
59     {
60         rb.AddForce(new Vector3(1f, 0f, 0f) * forceMagnitude); // D:
           right

```

```

61     }
62 }
63
64 private void OnTriggerEnter(Collider other)
65 {
66     if (other.gameObject.CompareTag("Coin"))
67     {
68         other.gameObject.SetActive(false);
69         points += 1;
70     }
71 }
72
73 public void LoadNextScene()
74 {
75     int currenSceneIndex = SceneManager.GetActiveScene().buildIndex;
76     int totalScenes = SceneManager.sceneCountInBuildSettings;
77
78     if (currenSceneIndex + 1 < totalScenes)
79     {
80         SceneManager.LoadScene(currenSceneIndex + 1);
81     }
82     else
83     {
84         Debug.Log("No more stages!");
85     }
86 }
87 }

```

Listing 3 FollowPlayer.cs

```

1 using UnityEngine;
2
3 public class FollowPlayer : MonoBehaviour
4 {
5     public Transform player;
6     private Vector3 offset;

```

```

7
8 // Start is called once before the first execution of Update after
  the MonoBehaviour is created
9 void Start()
10 {
11     offset = transform.position - player.position;
12 }
13
14 // Update is called once per frame
15 void LateUpdate()
16 {
17     transform.position = player.position + offset;
18 }
19 }

```

Listing 4 RotateCoin.cs

```

1 using UnityEngine;
2
3 public class RotateCoin : MonoBehaviour
4 {
5     public float rotationSpeed;
6     // Start is called once before the first execution of Update after
  the MonoBehaviour is created
7 void Start()
8 {
9
10 }
11
12 // Update is called once per frame
13 void FixedUpdate()
14 {
15     transform.Rotate(new Vector3(0f, 0f, 1f) * rotationSpeed);
16 }
17 }

```

B Unity Asset Store で入手した Asset

Unity Asset Store で入手したものであり, Asset の名前と URL を以下に示す.
`https://assetstore.unity.com/packages/3d/props/eight-ball-rack-24730`